

Riemann Assignment

Riemann is a **low-latency event stream processor** designed to monitor distributed systems. Unlike traditional monitoring tools that 'pull' data at intervals, Riemann is **push-based**, meaning it receives and processes thousands of events per second in real-time, allowing it to detect and react to system changes in milliseconds.

Events: The "Status Updates"

Think of an **Event** as a digital post-it note. Your servers are constantly scribbling notes and throwing them at Riemann.

host	A hostname, e.g. "api1", "foo.com"
service	e.g. "API port 8000 reqs/sec"
state	Any string less than 255 bytes, e.g. "ok", "warning", "critical"
time	The time of the event, in unix epoch seconds
description	Freeform text
tags	Freeform list of strings, e.g. ["rate", "fooproduct", "transient"]
metric	A number associated with this event, e.g. the number of reqs/sec.
ttl	A floating-point time, in seconds, that this event is considered valid for. Expired states may be removed from the index.

The Index:

Riemann has a big whiteboard called the **Index**. The index just keeps track of the most recent event for a given (host, service) pair.

- **The Rule:** The Index only keeps the **most recent** note for every unique service.
- If "Server A" sends a note saying "CPU is 10%" and then a minute later sends "CPU is 20%," the Index rubs out the first one and writes down the second.
- **Why it matters:** When you look at a Riemann Dashboard, you aren't seeing a history of everything that ever happened; you are seeing the **Index**—the "Current State" of your entire world right now.

TTL (Time To Live):

This is one of Riemann's smartest features. Every event has a "best before" date called a **TTL**.

- **Traditional Monitoring:** You check a server. If it doesn't answer, you know it's dead.
- **Riemann's Way:** The server says "I'm okay, and this note is valid for 60 seconds."
- **The "Expiry":** If 60 seconds pass and the server hasn't sent a *new* note, Riemann automatically marks that service as **"Expired."** It treats a "silence" exactly like an "error," allowing you to catch crashes instantly without constantly "polling" the server.

Streams: The "Sorting Machine"

If the Index is a whiteboard, **Streams** are the pipes that move the data around.

- **The Logic:** You write rules (in a language called Clojure) to tell the data where to go.
- **Example:** `(where (state "critical") (email "boss@company.com"))`
 - This acts like a **filter**. It looks at every "note" flying past. If the note says "critical," it triggers an email. If it doesn't, it just lets the note keep flowing down the pipe to the next rule.
- **Pipes, not Queries:** You don't "ask" Riemann for data like a database. You build a **pipeline**, and the data performs actions as it falls through the pipes.
- **A Continuous Pipeline:** Streams are not database queries; they are programmable "pipes." Events flow through them in real-time as they happen.
- **The Decision Engine:** Each stream acts as a filter or a logic gate. It examines an event (e.g., "Is this a critical error?") and decides whether to pass it to the next step, such as an alert or a dashboard.
- **Action-Oriented:** Streams allow you to **filter**, **transform**, and **combine** events. You can write rules to catch a single spike or aggregate data over time (like calculating an average) before taking an action like sending an email.

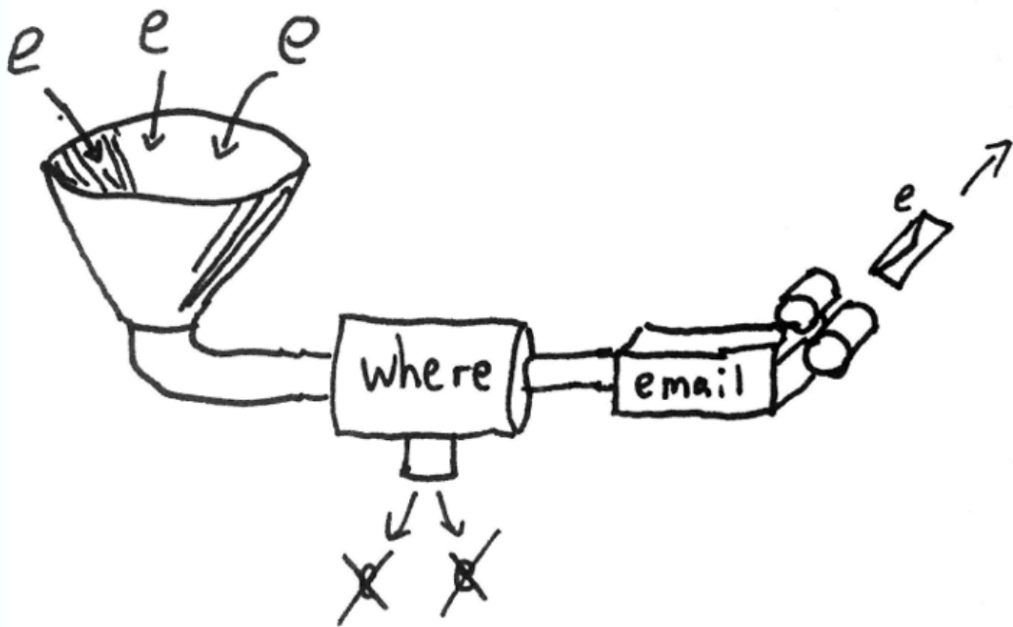
Here's a simple stream which sends critical events from any service beginning with "riak" to delacroix@vonbraun.com.

None

```
(streams  
  (where (and (service #"^riak"))
```

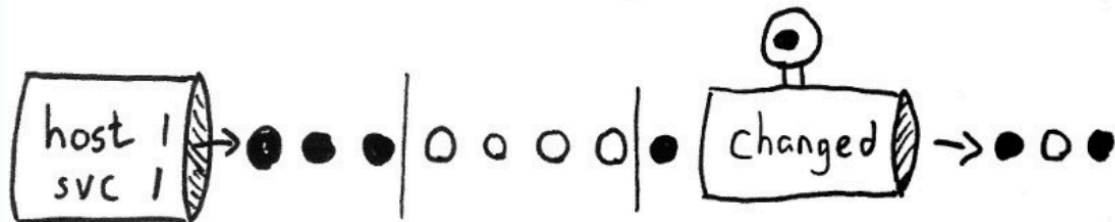
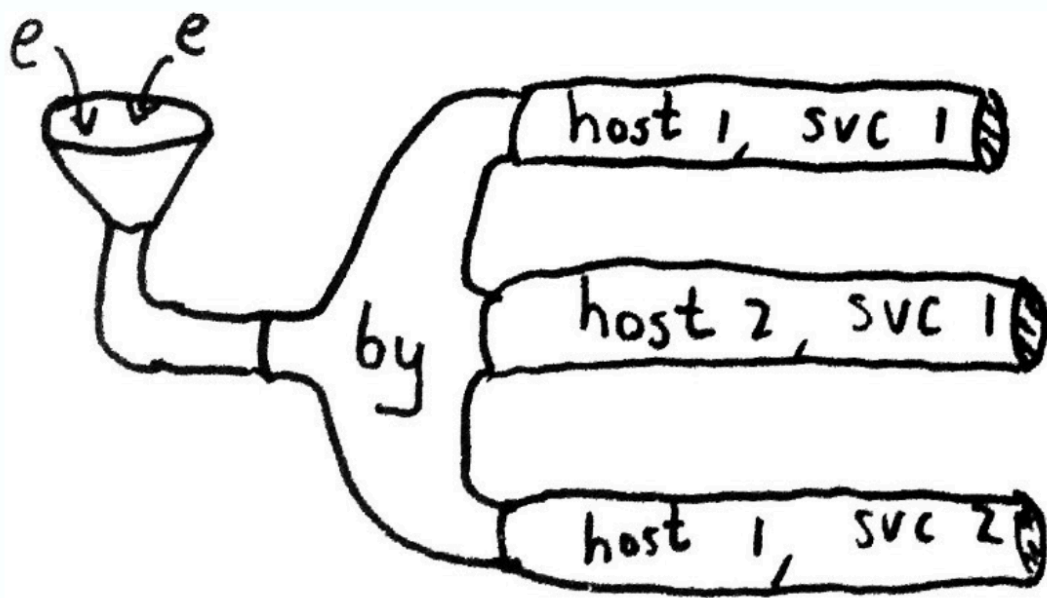
```
(state "critical"))
```

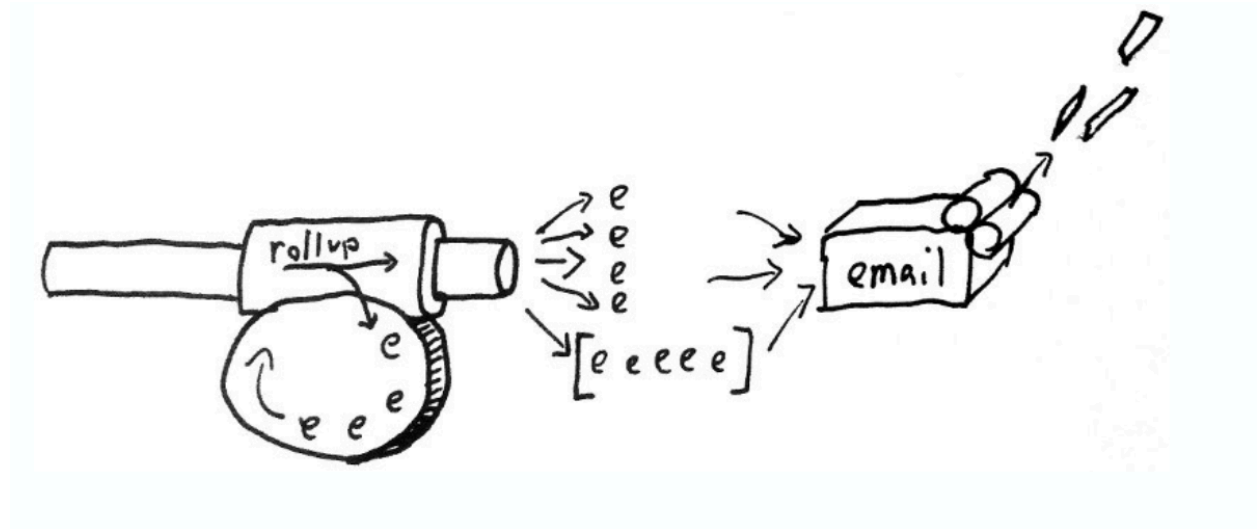
```
(email "delacroix@vonbraun.com"))))
```



None

```
(by [:host :service]
  (changed :state
    (rollup 5 3600
      (email "delacroix@vonbraun.com"))))
```





The Workflow: From Event to Alert

1. **Riemann Client (The Source):** Apps or small daemons (like `riemann-health`) **push** events. They are the "sensors" in your infrastructure.
2. **Riemann (The Brain):** This is your **Event Stream Processor**. It receives the data, applies logic (filtering, "is it broken?"), and decides where it needs to go next.
3. **InfluxDB (The Memory):** Riemann pushes metrics to InfluxDB. InfluxDB is a **Time-Series Database (TSDB)** optimized for storing historical data (e.g., "What was the CPU usage for the last 30 days?").
4. **Grafana (The Eyes):** Grafana connects to InfluxDB. It turns the raw numbers into beautiful, easy-to-read charts and dashboards for humans to look at.
5. **Alert Aggregator (The Megaphone):** If Riemann detects a "Critical" state, it pushes an alert here (e.g., PagerDuty, VictorOps, or Slack). The aggregator ensures the right person gets the notification.

Who monitors the monitor ?

Riemann features built-in **Instrumentation**. It generates internal events every second that track its own latency, memory usage, and throughput.

Because these internal metrics are treated as standard Riemann events, they flow through the same **Streams** as our application data. This means I can set up alerts to notify me if Riemann's

processing latency spikes or if its internal 'high-water marks' are reached. It essentially uses its own stream-processing engine to monitor its own health.

```
(instrumentation {:enabled? false}).
```

Based on your understanding of the monitoring stack

- identify what could break monitoring

When the data is not reaching the monitoring service
When the storage is not managed properly
When monitoring logic fails

- will you be able to identify when a component within the monitoring stack goes down. If so how

Riemann features built-in **Instrumentation**. It generates internal events every second that track its own latency, memory usage, and throughput.

InfluxDB is very similar to Riemann; it is highly self-aware.

InfluxDB has a built-in database called `_internal`. It automatically records how many points are being written per second, how much disk space is left, and how long queries are taking. It provides an `/health` or `/ping` HTTP endpoint. If you send a request to it and don't get a **200 OK** back, you know the database is hanging.

Grafana doesn't "push" events like Riemann, but it exposes its vitals. Grafana has a special URL (`/metrics`) that outputs data in a format other tools can read. It reports things like how many users are logged in, which dashboards are "heavy" and slow, and how many database connections it's using. **Internal Logs:** It is very vocal in its logs about "Data Source" errors (e.g., "I tried to talk to InfluxDB but it timed out").

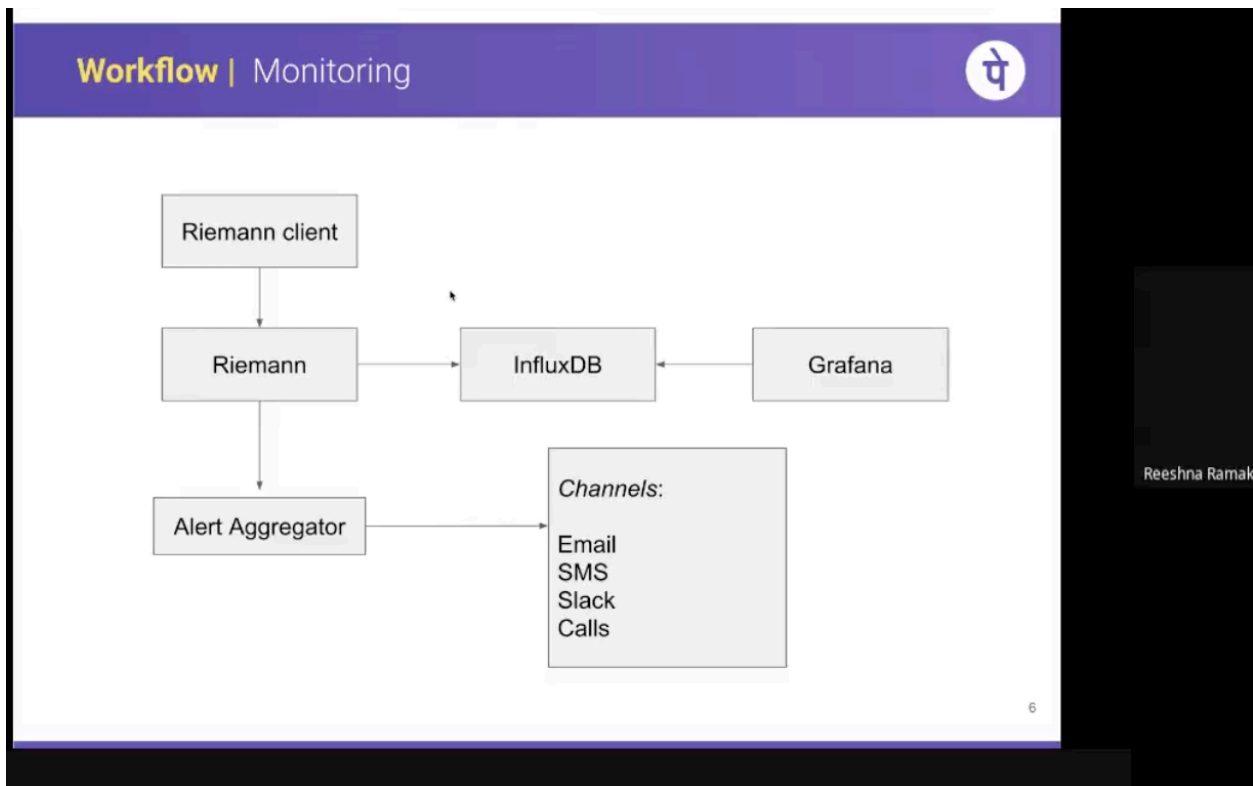
Since the clients (like `riemann-health`) are often just small scripts or daemons, they monitor the **Host**, but they also have a "Heartbeat." **TTL (Time To Live)** is the client's way of saying "I'm alive." If the client process crashes, it stops sending events. Riemann notices the **silence** and marks it as `expired`. This is "self-monitoring by proxy."

Tools like PagerDuty or Alerta have their own "Watchdog" systems. **Infrastructure Health:** They usually have a public status page. **API Limits:** They monitor if you are sending too many alerts too fast (Rate Limiting) and will actually send you a "System Alert" if your integration is broken.

We can also set a third party service to monitor the health of all the components in the monitoring stack.

- [how can you bring redundancy here](#)

You can give a writeup (use diagrams if you like to convey better)



Setup Riemann

1. bring up toy riemann server whose job is to print/log events to stdout as events come

Step 1.1: `multipass launch --name riemann-vm-2 --memory 2G --disk 10G`

Step 1.1: *multipass shell riemann-vm-2*

Step 1.2: *wget*

https://github.com/riemann/riemann/releases/download/0.3.9/riemann_0.3.9_all.deb

This is a downloader. We are telling the VM to reach out to the internet and download the specific installation file (`.deb`) for Riemann.

Step 1.3: *sudo dpkg -i riemann_0.3.9_all.deb*

This unpacks the Riemann software and puts the files in the right folders (like `/etc/riemann` and `/usr/bin`).

Step 1.4: *sudo vim /etc/riemann/riemann.config*

let [host "0.0.0.0"]: By default, servers only listen to themselves (`127.0.0.1`). Setting this to `0.0.0.0` tells Riemann: "Listen to any data coming from the outside world (like your dashboard or Python script)."

tcp/udp/ws-server: These are the **doors (ports)**. You are opening Port 5555 for data and Port 5556 for the web dashboard.

index: This creates a "whiteboard" where Riemann writes down the **latest** version of every event it hears.

prn: This is the most important part for your "Toy Server" requirement. It tells Riemann: "As soon as you hear a piece of data, print it out to the screen (stdout) immediately so I can see it."

```
None
; Listen on all interfaces
(let [host "0.0.0.0"]
  (tcp-server {:host host})
  (udp-server {:host host})
  (ws-server  {:host host})) ; Required for the Dashboard!
; Expire events every 5 seconds
(periodically-expire 5)
(let [index (index)]
  (streams
    (default :ttl 60
      index
      prn ; This satisfies the "Toy Server logs to stdout" requirement
    )))
```

Step 1.5: *sudo systemctl restart riemann*

Step 1.6: ***sudo systemctl status riemann***

```
[ubuntu@riemann-vm-2:~]$ sudo systemctl restart riemann
[ubuntu@riemann-vm-2:~]$ sudo systemctl status riemann
● riemann.service - LSB: Riemann server
   Loaded: loaded (/etc/init.d/riemann; generated)
   Active: active (exited) since Fri 2025-12-26 19:22:37 IST; 10s ago
     Docs: man:systemd-sysv-generator(8)
   Process: 1673 ExecStart=/etc/init.d/riemann start (code=exited, status=0/SUCCESS)
      CPU: 6ms

Dec 26 19:22:37 riemann-vm-2 systemd[1]: Starting riemann.service - LSB: Riemann server...
Dec 26 19:22:37 riemann-vm-2 systemd[1]: Started riemann.service - LSB: Riemann server.
```

2. write a plugin/script in a language of your choice to monitor server load avg every min and send as event to riemann (Clients are available - <https://riemann.io/clients.html>)

Step 2.1 : ***sudo apt update && sudo apt install -y python3-pip***

Step 2.2 : ***pip install bernhard --break-system-packages***

`pip install bernhard` ...: `bernhard` is a specific library that teaches Python how to speak "Riemann."

`--break-system-packages`: This is a safety override. Modern Ubuntu tries to stop you from installing things that might mess up the system Python, but here we are telling it, "It's okay, I know what I'm doing in this sandbox."

Step 2.3 : ***sudo vim monitor_load.py***

None

```
import os, time, bernhard, socket
client = bernhard.Client(host='127.0.0.1', port=5555)
while True:
    load = os.getloadavg()[0]
    client.send({'host': socket.gethostname(), 'service':
'load-avg', 'metric': float(load), 'ttl': 120})
    print(f"Sent load: {load}")
    time.sleep(60)
```

Step 2.4 : ***nohup python3 monitor_load.py > monitor.log 2>&1 &***

nohup ... **&**: Usually, if you close your terminal window, the script stops. **nohup** (No Hang Up) and the **&** symbol keep the script running in the background forever, even if you log out.

> monitor.log 2>&1: This saves any error messages or "Sent load" messages into a file called **monitor.log** so you can read them later.

Step 2.5 : ***pip install 'protobuf<4.0.0' --break-system-packages***

pip install 'protobuf<4.0.0' ...: This is a classic "debugging" step. A newer version of a background tool called **protobuf** broke the **Bernhard** library. By installing an older version (less than 4.0.0), you fixed the "TypeError" we saw earlier.

Step 2.6 : ***sudo apt update***

Step 2.7 : ***sudo apt install -y default-jre***

sudo apt install -y default-jre: Riemann is built using a language called Clojure, which requires Java to run. Your VM started empty, so this command installs the "Java Runtime Environment" (JRE)—the engine that powers Riemann.

java -version: This is just a quick check to make sure the engine is installed and working correctly.

Step 2.8 : ***java -version***

Step 2.9 : ***sudo systemctl restart riemann***

Step 2.10 : ***nohup python3 ~/monitor_load.py > monitor.log 2>&1 &***

Step 2.11 : ***ps aux | grep monitor_load.py***

Step 2.12: ***python3 ~/monitor_load.py &***

~~sent load: 0.02700200125~~

Sent load: 0.02099609375

Sent load: 0.11376953125

Sent load: 0.087890625

Sent load: 0.2607421875

Sent load: 0.2021484375

Sent load: 0.09423828125

Sent load: 0.07275390625

Sent load: 0.03271484375

Sent load: 0.02490234375

Sent load: 0.078125

Sent load: 0.060546875

Sent load: 0.02734375

Sent load: 0.18115234375

Sent load: 0.083984375

Sent load: 0.06494140625

Sent load: 0.029296875

Sent load: 0.02197265625

Sent load: 0.0087890625

Sent load: 0.14208984375

Sent load: 0.0651206875

[illegible]

3. setup riemann-dashboard that displays riemann's index (in memory state of current events) and create a dashboard to see load event being sent (<https://riemann.io/dashboard.html>)

Step 3.1: *sudo apt install rbenv*

We installed `rbenv`, which is like a Switcher. It allows us to have multiple versions of Ruby on one computer without them fighting each other.

Step 3.2: `mkdir -p "$(rbenv root)"/plugins`

This creates a "closet" inside the `rbenv` folder to hold extra tools.

```
Step 3.3: if [ -d "$(rbenv root)"/plugins/ruby-build ]; then
  cd "$(rbenv root)"/plugins/ruby-build && git pull
else
  git clone https://github.com/rbenv/ruby-build.git "$(rbenv root)"/plugins/ruby-build
fi
```

If you already have the `ruby-build` tool, it runs `git pull` to update the list of available Ruby versions.

Else (if you don't have it), it downloads (`git clone`) the latest version from GitHub.

`ruby-build` is the most important plugin for `rbenv`. It contains the "recipes" for every Ruby version ever released. Without this, you wouldn't be able to run `rbenv install 2.7.8` because your computer wouldn't know where to find that version or how to put it together.

Step 3.4: **`rbenv install 2.7.8`**

The newest Ruby (3.2+) removed some old features that the Riemann Dashboard needs to work. We installed version 2.7.8 because it is the "sweet spot" where everything still works perfectly.

Step 3.5: **`rbenv global 2.7.8`**

This tells the computer: "For the rest of this project, use this specific version of Ruby."

Step 3.6: **`echo 'export PATH="$HOME/.rbenv/bin:$PATH"' >> ~/.bashrc`**

The Details: `PATH` is a list of folders your computer searches whenever you type a command (like `ruby` or `ls`). By adding the `.rbenv/bin` folder to the front of that list, you are telling the computer: "Check my custom Ruby tools first before checking the standard system tools."

`>> ~/.bashrc`: This writes the instruction into a hidden file called `.bashrc`, which is the "Settings" menu for your terminal.

Step 3.7: **`echo 'eval "$(rbenv init -)"' >> ~/.bashrc`**

`rbenv init` is the command that handles the actual switching of Ruby versions. Without this line in your settings file, you would have to manually start `rbenv` every single time you logged in, which would be very annoying.

Step 3.8: **`source ~/.bashrc`**

Normally, changes to the `.bashrc` file only take effect when you close the terminal and open a new one. `source` is a shortcut that forces the current terminal to reload its settings immediately so you don't have to restart your session.

Step 3.9: **`which ruby`**

It confirms the computer is using the Ruby inside your `.rbenv` folder rather than the system default.

Step 3.10: **`gem update --system 3.4.22`**

Gems" are apps for Ruby. This command downloads and installs the actual Dashboard software.

Step 3.11: ***gem install riemann-dash -v 0.2.14***

Step 3.12: ***DASH_PATH=\$(gem which riemann/dash | sed 's/^.rb//')***

Step 3.13: ***cd***

/home/ubuntu/.rbenv/versions/2.7.8/lib/ruby/gems/2.7.0/gems/riemann-dash-0.2.14/lib/riemann/dash/views

Step 3.14: ***ln -sf index.erubis index.erb***

This is a clever fix for a common bug. The dashboard looks for files ending in `.erb`, but they were named `.erubis`. You created a "Symbolic Link" (like a desktop shortcut), so when the dashboard asks for `index.erb`, it is redirected to the correct file.

Step 3.15: ***ln -sf layout.erubis layout.erb***

Step 3.16: ***cd***

Step 3.17: ***sudo vim [config.rb](#)***

None

```
# /home/ubuntu/config.rb
set :bind, "0.0.0.0"
set :port, 4567
# Absolute paths to avoid any confusion
gem_dir =
"/home/ubuntu/.rbenv/versions/2.7.8/lib/ruby/gems/2.7.0/gems/riemann-dash-0.2.14/lib/riemann/dash"
set :views,          gem_dir + "/views"
set :public_folder, gem_dir + "/public"
# Disable logging to prevent the 'tainted?' error on newer Ruby environments
set :logging, false
```

~

Step 3.18: ***rbenv rehash***

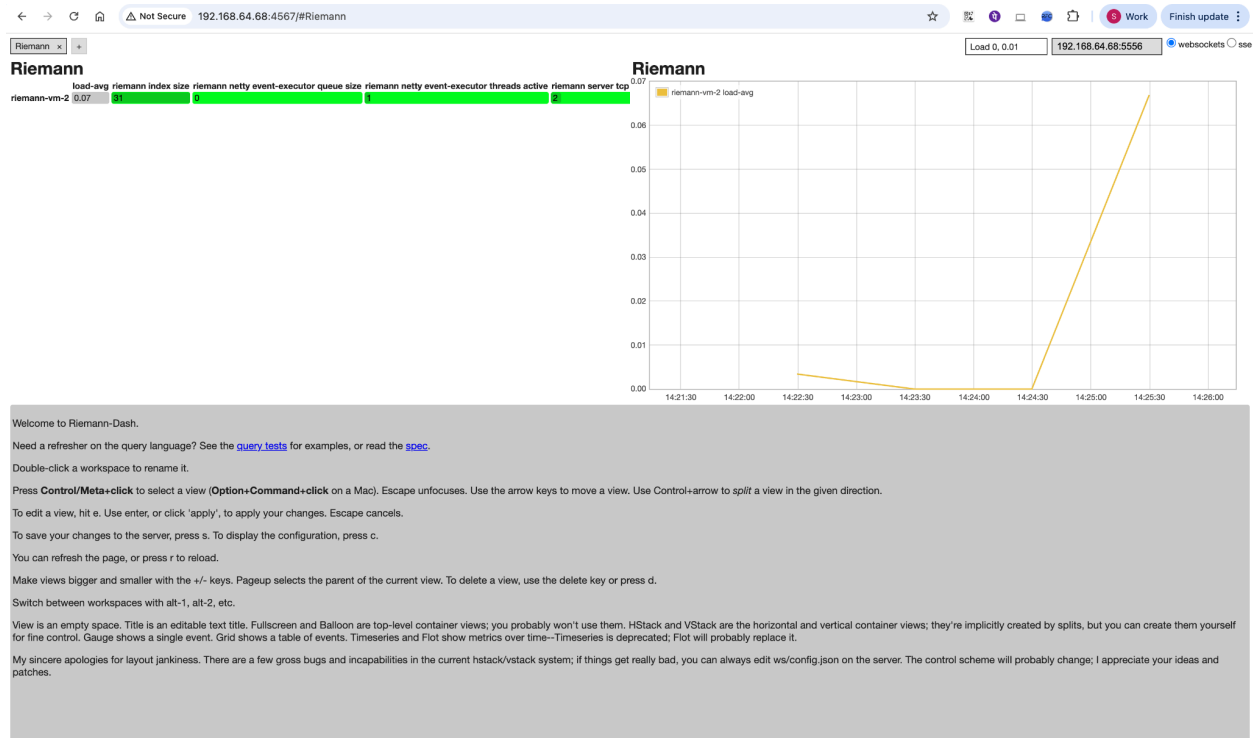
rbenv rehash: This refreshes the Switcher's memory so it recognizes the newly installed `riemann-dash` command.

Step 3.19: *which riemann-dash*

Step 3.20: *riemann-dash config.rb*

This starts the web server. It "plugs in" to Riemann's data feed on port 5556 and displays it on a website on port 4567.

Check the Dashboard: Open <http://IP:4567> (Shows the current **load-avg** value).



The Step-by-Step Data Journey

1. **The Server (The Source):** Your computer is running tasks. This creates "Load."
2. **The Client (The Messenger):** Your Python/Ruby script grabs that "Load" number. It wraps it in an event (like an envelope) and **pushes** it to Riemann.
3. **Riemann (The Indexer):** Riemann receives the envelope. It says, *"Okay, the load for 'Server-A' is now 0.5."* It writes this down in its **Index** (a fast, in-memory table of the "latest" values).
4. **The Dashboard (The Viewer):** When you open your browser, the dashboard asks Riemann: *"Give me everything in your index where the service is 'load-avg'."* Riemann sends the data, and the dashboard turns it into a pretty graph.

- See if you can combine Riemann server and Riemann dash installation as docker container

Step 4.1: ***mkdir riemann-docker && cd riemann-docker***

Step 4.2: ***sudo vim riemann.config***

```
None
; /riemann-docker/riemann.config
(let [host "0.0.0.0"]          // lets data inside the , will listen for every port
  (tcp-server {:host host})    // these are computer protocols
  (udp-server {:host host})
  (ws-server  {:host host}))  // this is for pushing data to out dashboard
(periodically-expire 5)      // expires data every 5 sec
(let [index (index)]         // saves the changes riemann makes on the dashboard
  (streams
    (default :ttl 60         // default validity of every data arriving
      index                  // save data on dashbaord
      prn)))                 // print data on the screen
```

Step 4.3: ***sudo vim [config.rb](#)***

```
None
set :bind, "0.0.0.0" // this tells dashboard to be visible to the world
set :port, 4567 //Port 4567 is the standard door for the software we are using (Sinatra).
set :logging, false // this is to prevent dashboard from crashing
```

Step 4.4: ***sudo vim docker-compose.yml***

```
None
version: '3.8'
services:
  # 1. Riemann Server (Custom build to support ARM64/Apple Silicon)
  riemann:
    build: .
    ports:
      - "5555:5555"
      - "5555:5555/udp"
      - "5556:5556"
    volumes:
      - ./riemann.config:/etc/riemann/riemann.config
```



```

    restart: always
# 2. Riemann Dashboard (Pivoted to Ruby 2.7 with manual dependency fixes)
dash:
  image: ruby:2.7
  ports:
    - "4567:4567"
  volumes:
    - ./config.rb:/app/config.rb
  working_dir: /app
  # Steps: Update RubyGems -> Install Dash -> Fix View Symlinks -> Run
  command: >
    sh -c "gem update --system 3.4.22 &&
    gem install riemann-dash -v 0.2.14 &&
    DASH_PATH=$(gem which riemann/dash | sed 's/\.rb//') &&
    ln -sf $$DASH_PATH/views/index.erubis $$DASH_PATH/views/index.erb &&
    ln -sf $$DASH_PATH/views/layout.erubis $$DASH_PATH/views/layout.erb &&
    riemann-dash config.rb"
  depends_on:
    - riemann
  restart: always
# 3. Python Monitor (Sending system load-avg every 60 seconds)
monitor:
  image: python:3.9-slim
  volumes:
    - ./monitor_load.py:/app/monitor_load.py
  working_dir: /app
  # Steps: Install client & compatible Protobuf -> Run script
  command: >
    sh -c "pip install bernhard 'protobuf<4.0.0' &&
    python monitor_load.py"
  depends_on:
    - riemann
  restart: always

```

Step 4.5: *sudo vim Dockerfile*

None

```

# Use a valid ARM64 compatible Java image
FROM eclipse-temurin:11-jre-focal
# Install wget to get the riemann package
RUN apt-get update && apt-get install -y wget && \
    wget https://github.com/riemann/riemann/releases/download/0.3.9/riemann_0.3.9_all.deb && \
    dpkg -i riemann_0.3.9_all.deb && \
    rm riemann_0.3.9_all.deb
# Expose Riemann ports
EXPOSE 5555 5555/udp 5556
# Start Riemann

```

```
CMD ["riemann", "/etc/riemann/riemann.config"]
```

Step 4.6: ***sudo vim monitor_load.py***

None

```
import os
import time
import bernhard
import socket
# Inside Docker, 'riemann' refers to the service name defined in docker-compose.yml
RIEMANN_HOST = 'riemann'
RIEMANN_PORT = 5555
def monitor_system_load():
    print(f"--- Starting Riemann Load Monitor (Target: {RIEMANN_HOST}) ---")
    # Initialize the Riemann Client
    try:
        client = bernhard.Client(host=RIEMANN_HOST, port=RIEMANN_PORT)
    except Exception as e:
        print(f"Could not connect to Riemann: {e}")
        return
    while True:
        try:
            # Get the 1-minute load average
            load = os.getloadavg()[0]
            # Create the event dictionary
            event = {
                'host': socket.gethostname(),
                'service': 'load-avg',
                'metric': float(load),
                'state': 'ok' if load < 1.0 else 'warning',
                'ttl': 120 # Time to Live: 120 seconds
            }
            # Send the event to Riemann
            client.send(event)
            print(f"Sent load: {load}")
        except Exception as e:
            print(f"Error sending event: {e}")
            # Wait for 60 seconds (1 minute) as per requirements
            time.sleep(60)
if __name__ == "__main__":
    monitor_system_load()
```

Step 4.7: ***sudo apt update***

Step 4.8: ***sudo apt install -y docker-compose***

Step 4.9: ***sudo usermod -aG docker \$USER***

Step 4.10: ***newgrp docker***

Step 4.11: *sudo docker-compose ps*

```
ubuntu@riemann-vn-2:~/riemann-docker$ sudo docker-compose ps

```

Name	Command	State	Ports
riemann-docker_dash_1	sh -c gem update --system ...	Up	0.0.0.0:4567->4567/tcp, :::4567->4567/tcp
riemann-docker_monitor_1	sh -c pip install bernhard ...	Up	
riemann-docker_riemann_1	/__cacert_entryptpoint.sh ri ...	Up	0.0.0.0:5555->5555/tcp, :::5555->5555/tcp, 0.0.0.0:5555->5555/udp, :::5555->5555/udp, 0.0.0.0:5556->5556/tcp, :::5556->5556/tcp

```
ubuntu@riemann-vn-2:~/riemann-docker$
```

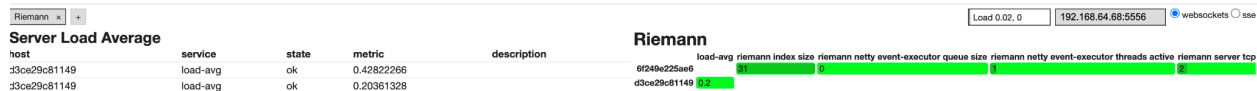
Step 4.12: *sudo docker-compose logs -f riemann*

```
ubuntu@riemann-vn-2:~/riemann-docker$ sudo docker-compose logs -f riemann
Attaching to riemann-docker_riemann_1
riemann_1 | INFO [2025-12-29 05:04:46,796] main - riemann.bin - Loading /etc/riemann/riemann.config
riemann_1 | INFO [2025-12-29 05:04:46,800] main - riemann.bin - PID 1
riemann_1 | INFO [2025-12-29 05:04:47,009] clojure-agent-send-off-pool-0 - riemann.transport.websockets - Websockets server 0.0.0.0 5556 online
riemann_1 | INFO [2025-12-29 05:04:47,191] clojure-agent-send-off-pool-1 - riemann.transport.tcp - TCP server 0.0.0.0 5555 online
riemann_1 | INFO [2025-12-29 05:04:47,231] clojure-agent-send-off-pool-0 - riemann.transport.udp - UDP server 0.0.0.0 5555 16384 -1 online
riemann_1 | INFO [2025-12-29 05:04:47,234] main - riemann.core - Hyperspace core online
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams rate", :state nil, :description nil, :metric 0.0, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.0", :state nil, :description nil, :metric nil, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.5", :state nil, :description nil, :metric nil, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.95", :state nil, :description nil, :metric nil, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.99", :state nil, :description nil, :metric nil, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.999", :state nil, :description nil, :metric nil, :tags ["riemann"], :time 883492348609/500, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann netty event-executor threads active", :state "ok", :description nil, :metric 1, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann netty event-executor queue size", :state "ok", :description nil, :metric 0, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann index size", :state "ok", :description nil, :metric 0, :tags ["riemann"], :time 144746174307/250, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 conn", :state "ok", :description nil, :metric 1, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in rate", :state "ok", :description nil, :metric 0.0, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in latency 0.0", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in latency 0.5", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in latency 0.95", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in latency 0.99", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server top 0.0.0.0:5555 in latency 0.999", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server udp 0.0.0.0:5555 in rate", :state "ok", :description nil, :metric 0.0, :tags ["riemann"], :time 353396939447/200, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server udp 0.0.0.0:5555 in latency 0.0", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 353396939447/200, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server udp 0.0.0.0:5555 in latency 0.5", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 353396939447/200, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server udp 0.0.0.0:5555 in latency 0.95", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 353396939447/200, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server udp 0.0.0.0:5555 in latency 0.99", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 353396939447/200, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 conn", :state "ok", :description nil, :metric 0, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 out rate", :state "ok", :description nil, :metric 0.0, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in rate", :state "ok", :description nil, :metric 0.0, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in latency 0.0", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in latency 0.5", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in latency 0.95", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in latency 0.99", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann server ws 0.0.0.0:5556 in latency 0.999", :state "ok", :description nil, :metric nil, :tags ["riemann"], :time 1766984697231/1000, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.0", :state nil, :description nil, :metric 2.953714243621832, :tags ["riemann"], :time 14135877659/8, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.5", :state nil, :description nil, :metric 0.632583, :tags ["riemann"], :time 14135877659/8, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.95", :state nil, :description nil, :metric 10.760890, :tags ["riemann"], :time 14135877659/8, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.99", :state nil, :description nil, :metric 0.078625, :tags ["riemann"], :time 14135877659/8, :ttl 20}
riemann_1 | #riemann.codec.Event{:host "56b668e369bc", :service "riemann streams latency 0.999", :state nil, :description nil, :metric 0.078625, :tags ["riemann"], :time 14135877659/8, :ttl 20}
```

Step 4.13: ***sudo docker-compose up -d --build***

```
ubuntu@riemann-vm-2:~/riemann-docker$ sudo docker-compose logs -f dash
Attaching to riemann-docker_dash_1
dash_1      | Updating rubygems-update
dash_1      | Successfully installed rubygems-update-3.4.22
dash_1      | Installing RubyGems 3.4.22
dash_1      | Successfully built RubyGem
dash_1      |   Name: bundler
dash_1      |   Version: 2.4.22
dash_1      |   File: bundler-2.4.22.gem
dash_1      | Bundler 2.4.22 installed
dash_1      | RubyGems 3.4.22 installed
dash_1      | Regenerating binstubs
dash_1      | Regenerating plugins
dash_1      |
dash_1      | # 3.4.22 / 2023-11-09
dash_1      |
dash_1      | ## Enhancements:
dash_1      |
dash_1      | * Update SPDX license list as of 2023-10-05. Pull request
dash_1      |   [#7040](https://github.com/rubygems/rubygems/pull/7040) by
dash_1      |   github-actions[bot]
dash_1      | * Remove unnecessary rescue. Pull request
dash_1      |   [#7109](https://github.com/rubygems/rubygems/pull/7109) by
dash_1      |   deivid-rodriguez
dash_1      | * Installs bundler 2.4.22 as a default gem.
dash_1      |
dash_1      | ## Bug fixes:
dash_1      |
dash_1      | * Handle empty array at built-in YAML serializer. Pull request
dash_1      |   [#7099](https://github.com/rubygems/rubygems/pull/7099) by hsb
dash_1      | * Ignore non-tar format `.gem` files during search. Pull request
dash_1      |   [#7095](https://github.com/rubygems/rubygems/pull/7095) by dearblue
dash_1      | * Allow explicitly uninstalling multiple versions of same gem. Pull
dash_1      |   request [#7063](https://github.com/rubygems/rubygems/pull/7063) by
dash_1      |   kstevens715
dash_1      |
dash_1      | ## Performance:
dash_1      |
dash_1      | * Avoid regexp match on every call to `Gem::Platform.local`. Pull request
dash_1      |   [#7104](https://github.com/rubygems/rubygems/pull/7104) by segiddins
dash_1      |
dash_1      | ## Documentation:
dash_1      |
dash_1      | * Get `Gem::Specification#extensions_dir` documented. Pull request
dash_1      |   [#7040](https://github.com/rubygems/rubygems/pull/7040) by
```

Check the Dashboard: Open <http://IP:4567> (Shows the current **load-avg** value).



Welcome to Riemann-Dash.

Need a refresher on the query language? See the [query tests](#) for examples, or read the [gpg](#).

Double-click a workspace to rename it.

Press **Control/Meta+click** to select a view (**Option+Command+click** on a Mac). Escape unfocuses. Use the arrow keys to move a view. Use Control-arrow to split a view in the given direction.

To edit a view, hit **e**. Use enter, or click 'apply', to apply your changes. Escape cancels.

To save your changes to the server, press **s**. To display the configuration, press **c**.

You can refresh the page, or press **r** to reload.

Make views bigger and smaller with the +/- keys. Pageup selects the parent of the current view. To delete a view, use the delete key or press **d**.

Switch between workspaces with alt-1, alt-2, etc.

View is an empty space. Title is an editable text title. Fullscreen and Balloon are top-level container views; you probably won't use them. HStack and VStack are the horizontal and vertical container views; they're implicitly created by splits, but you can create them yourself for fine control. Gauge shows a single event. Grid shows a table of events. Timeseries and Flot show metrics over time--Timeseries is deprecated; Flot will probably replace it.

My sincere apologies for layout jankiness. There are a few gross bugs and incapacities in the current hstack/vstack system; if things get really bad, you can always edit ws/config.json on the server. The control scheme will probably change; I appreciate your ideas and patches.

Before anything can "run," Docker needs to create the **Image**. It reads the **Dockerfile** to install Java and Riemann.

When you run **docker-compose up**, Docker creates the **Virtual Network** (the "wiring" between the containers) and sets up the **Volumes** (the "pipes" that let the containers see your **riemann.config**, **config.rb**, and **monitor_load.py** files).

Now the containers turn on. This is where the **Configuration Files** come into play:

- **Riemann** starts up and immediately reads the **riemann.config** to know which ports to open.
- **Dash** starts up, updates its tools, and reads **config.rb** to know how to display the website.
- **Monitor** starts up, installs its library, and begins running **monitor_load.py**.

Once those three steps are done, the files work together in a **loop**:

1. **monitor_load.py** (The Reporter) grabs data.
2. It sends it to **riemann.config** (The Brain).
3. **riemann.config** prints it to the screen and saves it.
4. **config.rb** (The Face) shows that saved data to your browser.